```
Stepper stepper(steps, 8, 9, 10, 11);
```

Our instance is named `stepper` although we could use almost anything for the name. The first parameter is the number of steps that our stepper motor is capable of in one full rotation. For our stepper, it has 1.8º steps, meaning that it would take 200 steps to complete a single rotation. A 7.2º step stepper motor would likewise have 360º / 7.2º or 50 total steps per revolution. In our earlier code, we declared the number of steps for our motor as the constant variable `steps` that we will use later in our code. The next four parameters, pins1–4, are the pin numbers connected to the drivers. Refer to the schematic for which pins go where.

setSpeed()

With our instance created, we need to set the rotational speed for our motor. The syntax for this function only has one parameter, as follows:

```
name.setSpeed(rpm)
```

The speed for our motor is set as rotations per minute. This value determines how fast the stepper moves from step to step and will naturally vary for each stepper motor. Setting this value too high for your motor may cause skipped steps or intermittent operation. Choose a speed appropriate for your application and/or motor.

Remember that the `setSpeed()` function does not actually move the motor—it only establishes the speed of the motor when our next function is called.

step()

The `step()` function is what actually makes the motor move. The syntax is simple, but there are some things we need to keep in mind.

```
name.step(steps)
```

The one parameter to pass to this function is the number of steps that we want to move the stepper motor. For example, with a 1.8º step motor, one step equals a 1.8º movement. Setting the number of steps to 50 will turn the motor 90º clockwise on our 200-step motor, or a quarter of a revolution, where -50 will turn the motor 90º counterclockwise. As you see, a positive integer will create a movement in one direction while a negative value sends it in the reverse direction.

Unlike something similar to the `analogWrite()` function, which you set and forget, `step()` will prevent the Arduino from doing anything else until it has completed its full move. This is not that big of a deal if we step the motor in small increments each time we call the `step()` function, but it quickly adds up. As a practical example, if we set the RPM speed to 10 RPM, our 200-step motor would take 6 seconds to complete 200 steps. In other words, if we used the statement `stepper.step(200)` then that would be 6 seconds that we will not be able to read inputs, communicate with other devices, or anything else. Instead it might be better to complete large rotations with a loop statement like the following code fragment:

```
for (int i=0; i<200; i++) {
  stepper.step(1);
  if (digitalRead(2) == HIGH) break;
}
```

In this example, we are incrementing the stepper motor 200 times in single-step increments to complete one revolution. Instead of calling a single 200-step movement by using a `for` loop, each